

TinyMouse

Design & Requirements Specification

Version 0.1 - Computational lineage for collaborative science

Fork analyses, not petabytes.

TinyMouse is a content-addressed system for building, executing, sharing, and navigating reproducible scientific analyses over large shared datasets.

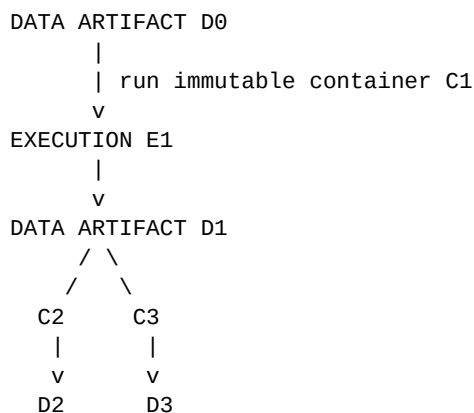
1. Executive Summary

TinyMouse treats scientific analysis as a graph of immutable data states connected by containerized transformations. A researcher begins from a registered dataset or derived artifact, runs code in an immutable OCI container, obtains a new content-addressed artifact, and records the complete provenance of that execution. Other researchers can jump to any verified point in the lineage and branch from there without copying the underlying dataset or rerunning unchanged upstream computation.

The system is intentionally not a replacement for Git. Git remains the source-of-truth for human-authored code and review history. TinyMouse adds a computational layer above it: containers package executable methods; a registry stores addressable scientific objects; and a lineage graph connects input data, code, environment, parameters, executions, and outputs.

The first implementation should be small: a Go CLI, a local Docker/Podman runner, an OCI-compatible registry, a metadata/provenance store, and a graph view. No Kubernetes is required for the MVP because analyses are batch computations: run once, preserve the result, and make it reusable.

Core idea



Each edge is an executed transformation.

Each node is immutable and content-addressed.

Researchers navigate only the local analysis branch they care about.

2. Product Goals and Non-Goals

Goals

- Allow many researchers to perform independent analyses over the same massive source dataset without duplicating it.
- Make every analysis reproducible by binding it to exact inputs, code, container image, parameters, and outputs.
- Represent analysis history as a navigable lineage graph rather than a flat repository or notebook collection.

- Allow researchers to branch from any derived artifact and reuse all valid upstream work.
- Deduplicate containers and artifacts through content-addressed storage.
- Make computational verification visible: users should be able to distinguish uploaded code from code actually executed against a declared data lineage.
- Keep the analysis language-agnostic: Python, R, Julia, C++, MATLAB-compatible runtimes, or other tools may run inside containers.
- Make the local path through analysis space easy to inspect while hiding unrelated branches by default.

Non-goals for v0.1

- High-availability application orchestration.
- Long-running services or microservice deployment.
- Replacing Git or GitHub.
- Building a new container runtime.
- Building a new blob storage protocol.
- Storing multi-terabyte source datasets inside Git repositories or container image layers.
- A general-purpose workflow language with every possible scheduler feature.

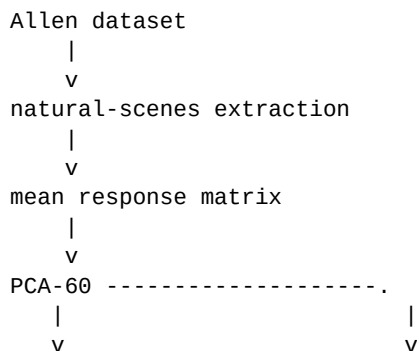
3. Conceptual Model

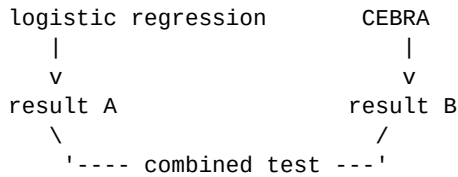
TinyMouse separates three graphs that are related but must not be conflated.

Graph	Owned by	Meaning
Code graph	Git	Human-authored source history, commits, branches, reviews.
Container graph	OCI/Docker	Reusable executable environments and filesystem layers.
Scientific graph	TinyMouse	Data -> execution -> derived data lineage and collaborative analysis paths.

Scientific graph

Internally, the scientific structure is a directed acyclic graph (DAG), not a strict tree. The user interface may present it as a tree-like local view because multiple analyses can share an intermediate artifact or reconverge on the same content-addressed result.





Analysis branch

An analysis branch is a user-facing concept: one coherent path through the DAG. It is not a Git branch. A branch may be shared, cited, forked from any node, compared with siblings, or extended.

4. Core Object Model

The MVP should recognize at least the following immutable object types:

Object	Purpose	Minimum identity
Dataset	Registered external or local source data.	URI/version + integrity digest or immutable provider identity
Artifact	Derived immutable data product.	Content digest
Container	Executable analysis environment.	OCI image digest
Code reference	Human-readable source provenance.	Git remote + commit SHA
Execution	A realized transformation.	Hash of input(s), container, parameters, command, contract version
Analysis branch	Named navigable path through executions.	Stable ID + ordered/linked lineage refs
Metadata record	Descriptions, labels, authorship, scientific notes.	Versioned record ID

Execution identity

A deterministic execution key should be derived from the exact computational inputs:

```

execution_key = H(
    input_artifact_digests,
    container_image_digest,
    normalized_parameters,
    entrypoint_or_command,
    TinyMouse_contract_version
)

```

If the execution key already exists and the prior result is valid, TinyMouse may return the cached output instead of recomputing it.

5. Container Execution Contract

TinyMouse should not care which language or scientific framework is inside a container. It only requires a small filesystem and process contract.

```

/input      read-only mounted input artifact(s)
/output     writable directory for declared outputs
/config     normalized parameters / execution metadata

```

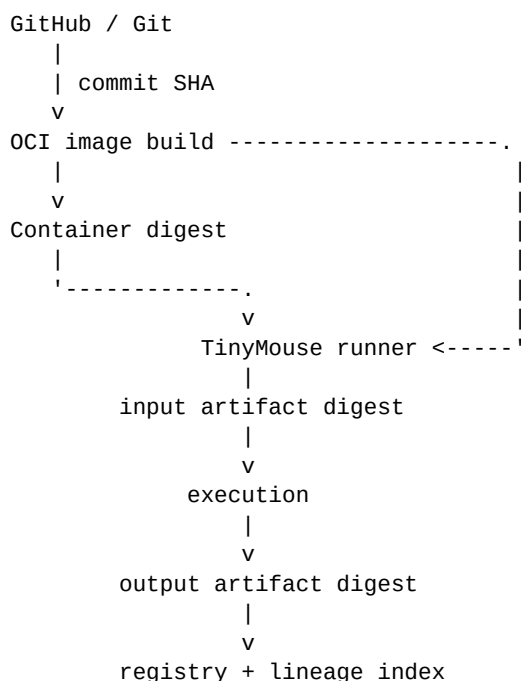
Container starts -> analysis runs once -> writes outputs -> exits.

Required behavior

- Inputs are mounted read-only.
- The container is executed by immutable digest, not by mutable tag alone.
- The container must write declared outputs to /output.
- Exit code 0 means computational success; non-zero means execution failure.
- TinyMouse hashes outputs after successful execution.
- TinyMouse records stdout/stderr as execution logs.
- The container process may disappear after completion; the scientific artifact and provenance persist.
- Network access should be disabled by default in a later hardened mode, with explicit opt-in when needed.

6. Registry and Storage Architecture

The registry is the computational memory of TinyMouse. Git stores source; the registry stores immutable executable and scientific state. OCI registries are attractive because they already provide content-addressed blobs, manifests, deduplication, image distribution, and established authentication patterns.



Large scientific data should normally live in object storage or an existing scientific archive.

TinyMouse registers and verifies those objects rather than copying them into Git or baking them into containers.

Container inheritance and deduplication

Analysis images may share OCI layers. A common scientific base image can contain CUDA/Python/scientific libraries, while child images add only analysis-specific code. Registries deduplicate shared layer blobs automatically. This optimization is independent of scientific lineage: two analyses may share container layers without sharing scientific ancestors, and vice versa.

7. Provenance Verification

TinyMouse should distinguish mere code availability from demonstrated execution against the declared data lineage.

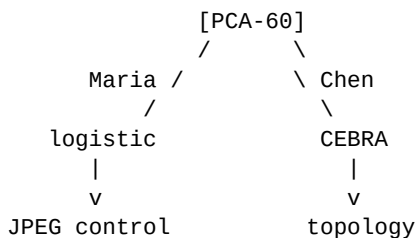
Status	Meaning
Unverified	Code or manifest exists, but TinyMouse has not observed a successful execution.
Executed	Container completed successfully and produced declared outputs.
Lineage verified	Every input can be traced to a registered source dataset or verified derivative.
Reproduced	An independent rerun produced matching outputs within declared tolerances.

The first MVP needs only Unverified, Executed, and Lineage verified. Independent reproduction can follow later.

8. User Experience

Local lineage view

The full DAG may become enormous. TinyMouse must therefore default to a local, navigable view centered on the current analysis path, its immediate ancestors, and nearby alternative descendants.



Current view: Maria branch
Nearby alternative: Chen branched at PCA-60
Everything else hidden until requested.

Core interactions

- Open a dataset and see major analysis families.
- Open an analysis branch and walk backward to its source data.
- Branch from any verified artifact.
- See where another researcher's analysis diverged from the current branch.
- Compare siblings that share the same input artifact.

- Filter to verified/reproduced analyses.
- Inspect the exact container, code commit, parameters, logs, and outputs of any execution.
- Copy a stable citation/reference for an analysis node.
- Search by dataset, author, method, output type, or scientific annotation.

9. Proposed CLI v0.1

```
tinymouse init
```

```
tinymouse dataset add <uri> [--digest ...]
tinymouse dataset inspect <ref>
```

```
tinymouse run \
  --input <artifact-ref> \
  --image <oci-image@sha256:...> \
  --param key=value \
  [--name "PCA 60"]
```

```
tinymouse lineage <artifact-or-execution-ref>
tinymouse branch <artifact-ref> --name <branch-name>
tinymouse status <ref>
tinymouse inspect <ref>
tinymouse push <ref>
tinymouse pull <ref>
```

The CLI should initially favor explicitness over convenience. Automatic Dockerfile builds, Git integration, remote workers, and AI-assisted analysis discovery can be layered on later.

10. Functional Requirements

ID	Requirement
FR-01	Register a source dataset or immutable external dataset reference.
FR-02	Register/pull an OCI container by digest.
FR-03	Execute a container once against one or more registered input artifacts.
FR-04	Mount inputs read-only and provide a writable output directory.
FR-05	Capture command, parameters, code reference, image digest, inputs, outputs, logs, timestamps, and exit status.
FR-06	Hash and register every produced output artifact.
FR-07	Create lineage edges from input artifact(s) through execution to output artifact(s).
FR-08	Detect an identical prior execution key and support cache reuse.
FR-09	Allow a user to create a named analysis branch from any artifact.
FR-10	Display ancestors and immediate alternative descendants of the current branch.
FR-11	Persist metadata separately from large raw data blobs.

FR-12	Export/import lineage manifests without copying external datasets.
FR-13	Associate execution images with Git commit provenance when available.
FR-14	Expose verification status for each execution/branch.

11. Non-Functional Requirements

ID	Requirement
NFR-01 Reproducibility	Every executed node must be resolvable to immutable inputs and an immutable container digest.
NFR-02 Portability	MVP must run on a single workstation or server with Docker or Podman.
NFR-03 Language neutrality	No scientific programming language may be required by the core.
NFR-04 Deduplication	Content-addressed objects and OCI layers should be reused when identical.
NFR-05 Scalability of metadata	Lineage traversal must remain practical when graphs contain many thousands of executions.
NFR-06 Large-data awareness	TinyMouse must not require cloning massive source datasets into each analysis workspace.
NFR-07 Auditability	Users must be able to inspect how each output was produced.
NFR-08 Failure isolation	A failed analysis must not mutate registered input artifacts.
NFR-09 Deterministic identity	Equivalent normalized execution specifications should map to the same execution key.
NFR-10 Extensibility	Storage backends, runners, registries, and metadata stores should be replaceable behind interfaces.

12. MVP Component Architecture

```

tinymouse (Go CLI)
|
|-- manifest/model package
|     immutable IDs, schemas, validation
|
|-- runner interface
|     Docker first; Podman adapter later
|
|-- artifact store interface
|     local filesystem first
|     object storage later
|
|-- registry interface
|     OCI-compatible registry
|
|-- lineage store
|     SQLite first
|     graph DB optional later
|
|-- Git adapter
|     detect repo URL + commit SHA

```


SQLite is sufficient for the first lineage index because the DAG edges and object metadata are small compared with the scientific artifacts themselves. A graph database should not be introduced until traversal or collaboration requirements justify it.

13. Example Execution Manifest

```
apiVersion: tinymouse.dev/v1alpha1
kind: Execution

metadata:
  name: pca-60-natural-scenes

inputs:
  - ref: tm://artifact/sha256:DATA...

runtime:
  image: ghcr.io/example/pca@sha256:IMAGE...
  command: ["python", "/app/run.py"]

source:
  git:
    remote: https://github.com/example/allen-analysis
    commit: 61ae9f...

parameters:
  n_components: 60
  center: true

outputs:
  - name: scores
    path: /output/pca_scores.zarr

provenance:
  parentBranch: tm://branch/animacy-base
```

14. Integrity and Security Boundaries

- Never trust mutable image tags as sufficient provenance; resolve and record digests.
- Mount registered inputs read-only.
- Do not execute arbitrary untrusted containers with unrestricted host access.
- Avoid mounting the Docker socket into analysis containers.
- Record image and artifact digests before/after execution as appropriate.
- Support signed images/artifacts later, but do not make signing a blocker for the first local prototype.
- Treat secrets as execution-time inputs that are never embedded in lineage manifests.
- Keep provenance records immutable; corrections should create superseding metadata rather than rewrite historical executions.

15. Implementation Plan

Milestone 1 - The tiny mouse actually runs

- Go module and CLI skeleton.
- Local content store.
- Artifact hashing.
- Docker runner.
- Minimal execution manifest.
- Record input -> execution -> output in SQLite.
- ``tinymouse lineage`` prints the ancestry as text.

Milestone 2 - Branching analyses

- Named branches.
- Branch-from-artifact command.
- Sibling/descendant discovery.
- Execution cache.
- Git commit auto-detection.
- Simple local web graph viewer.

Milestone 3 - Shared registry

- OCI registry push/pull.
- Remote artifact storage.
- Collaborator identities and permissions.
- Portable lineage manifests.
- Verified execution badges.

Milestone 4 - Allen-scale demonstration

- Register an Allen dataset/session without copying the complete source dataset.
- Create a containerized extraction step.
- Create at least two downstream analysis branches from the same derived artifact.
- Demonstrate cache reuse and OCI layer deduplication.
- Have a second researcher branch from an existing verified intermediate result.

16. MVP Acceptance Test

TinyMouse v0.1 is real when the following end-to-end scenario works:

1. Register D0: an immutable source or fixture dataset.
2. Run C1 against D0 -> produce D1.
3. Run C2 against D1 -> produce D2.
4. Run C3 against D1 -> produce D3.
5. ``tinymouse lineage D2`` shows D0 -> E1 -> D1 -> E2 -> D2.
6. The UI/CLI shows D3 as a sibling branch from D1.
7. Re-requesting E2 detects the same execution key and reuses D2.
8. Changing code/container/parameters produces a new execution identity.

9. D0 and D1 are never modified.
10. Another machine can resolve the same lineage from shared registry metadata.

17. Design Principles

Fork analyses, not petabytes. Collaboration should reuse shared data lineage instead of duplicating massive datasets.

Run once; preserve forever. The durable unit is the result plus provenance, not the living container process.

Content before names. Human names are convenient; digests determine identity.

Git for source, OCI for execution, TinyMouse for scientific lineage. Do not make one subsystem impersonate the others.

The graph may be huge; the view must be small. Show the current path and nearby alternatives by default.

A result is stronger when it has run. Executed and lineage-verified analyses deserve visible computational status.

18. Open Questions After v0.1

- How should external datasets be given stable identities when the upstream provider does not expose content digests?
- How should numeric reproducibility be declared when GPU kernels or floating-point operations are nondeterministic?
- Should analysis outputs be stored as generic OCI artifacts, in object storage, or through a hybrid manifest/blob model?
- What is the smallest metadata schema that scientists will actually maintain?
- How should private, embargoed, or access-controlled datasets participate in shared lineage graphs?
- When two branches produce identical artifact content from different execution paths, should the UI explicitly display reconvergence?
- What should constitute a citable TinyMouse analysis identifier?
- Which parts of the provenance record should be automatically inferred versus explicitly declared?

19. Working Product Statement

TinyMouse is a collaborative, content-addressed graph of scientific computation: immutable data artifacts are transformed by reproducible OCI containers, stored with verified lineage, and exposed as navigable analysis branches that other researchers can inspect, reuse, and extend.

Novelty note: this document is a product/architecture specification, not a patentability or prior-art opinion. The proposed combination should be evaluated against existing workflow, provenance, registry, and collaborative science systems before making formal novelty claims.